



Tecnológico de Monterrey

Evidencia 1 - Actividad Integral de Conceptos Básicos y Algoritmos Fundamentales (Evidencia Competencia)

VIDEO: <https://youtu.be/qD-fMYgEtMg>

CÓDIGO FUENTE: <https://github.com/ramondeleonca/ActIntAlg>

Ramón de León Cabrera

Prof. David Alonso Cantú Delgado

Plan de pruebas

Se diseñó un script de pruebas test.sh en el cual se ejecutan 6 pruebas “known-good” de ordenado y búsqueda por fecha para validar el funcionamiento correcto y esperado.

Script:

```
1  #!/bin/bash
2
3  # Compilar primero
4  g++ -std=c++20 src/main.cpp -o out/main
5
6  # Correr programa con un Here-Document (EOF) que inyecta
   automáticamente las combinaciones:
7  ./out/main << EOF
8  1
9  1
10 2000
11 2024:09:08-00:22:43
12 2024:09:08-05:50:57
13 1
14 2
15 1000
16 2024:09:08-00:22:43
17 2024:09:08-05:50:57
18 1
19 3
20 20
21 2024:09:08-00:22:43
22 2024:09:08-05:50:57
23 2
24 1
25 1500
26 2024:09:08-00:22:43
27 2024:09:08-05:50:57
28 2
29 2
30 10
31 2024:09:08-00:22:43
32 2024:09:08-05:50:57
33 2
34 3
35 20
36 2024:09:08-00:22:43
37 2024:09:08-05:50:57
38 0
39 EOF
40
```

Resultado:

```
→ ActIntAlg git:(main) x ./scripts/test.sh
=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir):
Algoritmos:
1. Bubble Sort
2. Insertion Sort
3. Merge Sort
Elige un algoritmo:
Prediccion (Escribe en milisegundos cuanto crees que tardara):
[Archivo: data/log607-1.txt | Tamano: 6818 registros]
[Prediccion: 2000 ms]
Algoritmo: Bubble Sort
Tiempo de ejecucion: 1616 ms
Error de prediccion: 384 ms
Complejidad teorica: O(N) mejor caso, O(N^2) peor caso
Datos ordenados guardados en output607.txt

=== Busqueda por rango ===
Ingresa la fecha/hora de INICIO (ej. 2024:09:08-00:22:43) (YYYY:MM:DD-HH:MM:SS): Ingresa la fecha/hora de
FIN (ej. 2024:09:08-05:50:57) (YYYY:MM:DD-HH:MM:SS): Registros encontrados en el rango: 4
Resultados guardados en range607.txt

=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir):
Algoritmos:
1. Bubble Sort
2. Insertion Sort
3. Merge Sort
Elige un algoritmo:
Prediccion (Escribe en milisegundos cuanto crees que tardara):
[Archivo: data/log607-1.txt | Tamano: 6818 registros]
[Prediccion: 1000 ms]
Algoritmo: Insertion Sort
Tiempo de ejecucion: 994 ms
Error de prediccion: 6 ms
Complejidad teorica: O(N) mejor caso, O(N^2) peor caso
Datos ordenados guardados en output607.txt

=== Busqueda por rango ===
Ingresa la fecha/hora de INICIO (ej. 2024:09:08-00:22:43) (YYYY:MM:DD-HH:MM:SS): Ingresa la fecha/hora de
FIN (ej. 2024:09:08-05:50:57) (YYYY:MM:DD-HH:MM:SS): Registros encontrados en el rango: 4
Resultados guardados en range607.txt

=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir):
Algoritmos:
```

```
→ ActIntAlg git:(main) ✗ ./scripts/test.sh
1. Bubble Sort
2. Insertion Sort
3. Merge Sort
Elige un algoritmo:
Prediccion (Escribe en milisegundos cuanto crees que tardara):
[Archivo: data/log607-1.txt | Tamano: 6818 registros]
[Prediccion: 20 ms]
Algoritmo: Merge Sort
Tiempo de ejecucion: 23 ms
Error de prediccion: 3 ms
Complejidad teorica:  $O(N \log N)$  mejor y peor caso
Datos ordenados guardados en output607.txt

=== Busqueda por rango ===
Ingresa la fecha/hora de INICIO (ej. 2024:09:08-00:22:43) (YYYY:MM:DD-HH:MM:SS): Ingresa la fecha/hora de
FIN (ej. 2024:09:08-05:50:57) (YYYY:MM:DD-HH:MM:SS): Registros encontrados en el rango: 4
Resultados guardados en range607.txt

=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir):
Algoritmos:
1. Bubble Sort
2. Insertion Sort
3. Merge Sort
Elige un algoritmo:
Prediccion (Escribe en milisegundos cuanto crees que tardara):
[Archivo: data/log607-2.txt | Tamano: 6818 registros]
[Prediccion: 1500 ms]
Algoritmo: Bubble Sort
Tiempo de ejecucion: 6 ms
Error de prediccion: 1494 ms
Complejidad teorica:  $O(N)$  mejor caso,  $O(N^2)$  peor caso
Datos ordenados guardados en output607.txt

=== Busqueda por rango ===
Ingresa la fecha/hora de INICIO (ej. 2024:09:08-00:22:43) (YYYY:MM:DD-HH:MM:SS): Ingresa la fecha/hora de
FIN (ej. 2024:09:08-05:50:57) (YYYY:MM:DD-HH:MM:SS): Registros encontrados en el rango: 4
Resultados guardados en range607.txt

=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir):
Algoritmos:
1. Bubble Sort
2. Insertion Sort
3. Merge Sort
Elige un algoritmo:
Prediccion (Escribe en milisegundos cuanto crees que tardara):
```

```
[Archivo: data/log607-2.txt | Tamano: 6818 registros]
[Prediccion: 10 ms]
Algoritmo: Insertion Sort
Tiempo de ejecucion: 2 ms
Error de prediccion: 8 ms
Complejidad teorica:  $O(N)$  mejor caso,  $O(N^2)$  peor caso
Datos ordenados guardados en output607.txt

=== Busqueda por rango ===
Ingrese la fecha/hora de INICIO (ej. 2024:09:08-00:22:43) (YYYY:MM:DD-HH:MM:SS): Ingrese la fecha/hora de
FIN (ej. 2024:09:08-05:50:57) (YYYY:MM:DD-HH:MM:SS): Registros encontrados en el rango: 4
Resultados guardados en range607.txt

=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir):
Algoritmos:
1. Bubble Sort
2. Insertion Sort
3. Merge Sort
Elige un algoritmo:
Prediccion (Escribe en milisegundos cuanto crees que tardara):
[Archivo: data/log607-2.txt | Tamano: 6818 registros]
[Prediccion: 20 ms]
Algoritmo: Merge Sort
Tiempo de ejecucion: 20 ms
Error de prediccion: 0 ms
Complejidad teorica:  $O(N \log N)$  mejor y peor caso
Datos ordenados guardados en output607.txt

=== Busqueda por rango ===
Ingrese la fecha/hora de INICIO (ej. 2024:09:08-00:22:43) (YYYY:MM:DD-HH:MM:SS): Ingrese la fecha/hora de
FIN (ej. 2024:09:08-05:50:57) (YYYY:MM:DD-HH:MM:SS): Registros encontrados en el rango: 4
Resultados guardados en range607.txt

=== Actividad Integradora ===
1. log607-1.txt (Desordenado)
2. log607-2.txt (Casi ordenado)
Elige un archivo (1/2, 0 para salir): %
```

Reflexión

1. Importancia de ordenar y buscar

Un archivo de logs de seguridad como log607-1.txt o log607-2.txt crece de forma continua y casi nunca llega ordenado por completo, porque los eventos se generan desde distintas fuentes que no siempre reportan en el mismo instante en que ocurren. Y casi cualquier análisis útil sobre estos datos depende de poder ubicar rápido un rango de fechas, si los datos no están ordenados, la única opción es una búsqueda lineal que revisa las 6,818 líneas una por una, cosa que en un log real de millones de entradas sería inviable para un analista que necesita responder a un incidente en minutos. Ordenar una sola vez y después buscar con búsqueda binaria cambia por completo el costo del problema: pasamos de $O(n)$ por cada consulta de rango a $O(\log n)$.

2. Comparación de los algoritmos

El proyecto implementó tres algoritmos de ordenamiento (Bubble Sort, Insertion Sort y Merge Sort) sobre la clase LogEntry, comparando por el campo dateTimeKey (string de la suma de YYYYMMDDHHMMSS).

Bubble Sort compara pares adyacentes y los intercambia si están en el orden incorrecto, repitiendo pasadas hasta que ya no hay cambios. Lo bueno es la simplicidad de implementación, lo malo es que en el peor caso (datos en orden inverso) hace $O(n^2)$ comparaciones e intercambios, lo que lo hace muy lento.

Insertion Sort construye la lista ordenada insertando cada elemento nuevo en su posición correcta dentro de la parte ya ordenada. También es $O(n^2)$ en el peor caso, pero tiene una ventaja práctica sobre Bubble Sort: hace menos movimientos de datos en promedio y es el estándar cuando se sabe que los datos llegan "**casi ordenados**".

Merge Sort divide la lista a la mitad recursivamente y después combina las dos mitades ya ordenadas. Garantiza $O(n \log n)$ tanto en el mejor como en el peor caso, a costo de usar memoria adicional (los vectores leftList y rightList en cada merge) y de ser mucho más complejo de implementar. Es la opción correcta cuando el tamaño del log es grande y no se puede garantizar nada sobre el orden de llegada de los datos, algo que yo veo muy útil.

La diferencia más interesante aparece con datos casi ordenados (log607-2.txt). Tanto Bubble Sort como Insertion Sort son algoritmos adaptativos: su condición de paro depende de si hubo o no intercambios necesarios. Si los datos ya están casi en orden, Bubble Sort deja de hacer pasadas completas casi de inmediato e Insertion Sort casi no necesita mover elementos hacia atrás, así que ambos se acercan a su mejor caso, $O(n)$, pero Merge Sort, en cambio, no es adaptativo, sin importar que tan ordenados estén los datos, siempre hace la misma cantidad de divisiones y de comparaciones.

3. Interpretación de las Mediciones

Se corrieron los tres algoritmos sobre los dos archivos (6,818 registros cada uno), comparando la prediccion inicial contra el tiempo real medido con `std::chrono`:

Archivo `log607-1.txt` (desordenado):

- Bubble Sort: prediccion 2000 ms, medido 3973 ms (error de 1973 ms)
- Insertion Sort: prediccion 1000 ms, medido 3522 ms (error de 2522 ms)
- Merge Sort: prediccion 20 ms, medido 44 ms (error de 24 ms)

Archivo `log607-2.txt` (casi ordenado):

- Bubble Sort: prediccion 1500 ms, medido 9 ms (error de 1491 ms)
- Insertion Sort: prediccion 10 ms, medido 6 ms (error de 4 ms)
- Merge Sort: prediccion 20 ms, medido 39 ms (error de 19 ms)

La conclusion general es que la complejidad teorica O describe el peor caso, pero el comportamiento real depende mucho de la estructura de los datos de entrada, y eso es exactamente lo que separa a un algoritmo adaptativo de uno que no lo es.

4. Uso de IA

Los tres prompts principales utilizados fueron:

Prompt 1:

Generame un plan sobre como debería realizar una busqueda binaria para encontrar una entrada al inicio y otra al final de un rango de fechas

Prompt 2:

Adapta las funciones de búsqueda en los archivos de la actividad 1.5 para funcionar con `LogEntries`

Prompt 3:

Completa un test automatizado en un bash script que corra el binario de `main` con diferentes parámetros

Caso concreto donde la IA se equivocó: al pedir la primera version de `binarySearchUpperBound`, la sugerencia inicial uso la condicion `list[mid].datetimeKey >= key` (la misma condicion que el límite inferior) en vez de una comparación estrictamente mayor. Así, `upperIdx` apuntaba al primer registro igual a la fecha de fin en lugar del primero posterior, y como el rango se recorre como `[lowerIdx, upperIdx)` en `saveRangeToFile`, el ultimo registro cuya fecha coincidía exactamente con la fecha de fin ingresada quedaba excluido del resultado: un error de límites off-by-one que sería muy fácil que también le pasara a un humano.